

C01 - Lab Exercises

```
# =====  
# 1. FIND-S ALGORITHM  
# Google Colab  
# =====  
  
import pandas as pd  
  
# -----  
# Step 1: Create Training Dataset  
# -----  
  
data = {  
    "Sky": ["Sunny", "Sunny", "Rainy", "Sunny"],  
    "AirTemp": ["Warm", "Warm", "Cold", "Warm"],  
    "Humidity": ["Normal", "High", "High", "High"],  
    "Wind": ["Strong", "Strong", "Strong", "Strong"],  
    "Water": ["Warm", "Warm", "Warm", "Cool"],  
    "Forecast": ["Same", "Same", "Change", "Change"],  
    "EnjoySport": ["Yes", "Yes", "No", "Yes"]  
}  
  
df = pd.DataFrame(data)  
  
print("TRAINING DATA")  
print("=" * 60)  
print(df)  
  
# -----  
# Step 2: FIND-S Algorithm  
# -----  
  
attributes = list(df.columns[:-1])  
  
# Initialize hypothesis with the most specific values  
hypothesis = ["Ø"] * len(attributes)  
  
print("\nInitial Hypothesis:")  
print(hypothesis)  
  
# Process each training example
```

```

for index, row in df.iterrows():

    # Consider only positive examples
    if row["EnjoySport"] == "Yes":

        for i, attribute in enumerate(attributes):

            # First positive example
            if hypothesis[i] == "∅":
                hypothesis[i] = row[attribute]

            # Generalize hypothesis when values differ
            elif hypothesis[i] != row[attribute]:
                hypothesis[i] = "?"

            print(
                f"\nAfter Positive Example {index + 1}:"
            )
            print(hypothesis)

# -----
# Step 3: Final Hypothesis
# -----

print("\nFINAL MOST SPECIFIC HYPOTHESIS")
print("=" * 60)

for attribute, value in zip(attributes, hypothesis):
    print(f"{attribute}: {value}")

```

Untitled4.ipynb_

[6]

0s

TRAINING DATA

	Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
0	Sunny	Warm	Normal	Strong	Warm	Same	Yes
1	Sunny	Warm	High	Strong	Warm	Same	Yes

2	Rainy	Cold	High	Strong	Warm	Change	No
3	Sunny	Warm	High	Strong	Cool	Change	Yes

Initial Hypothesis:

['Ø', 'Ø', 'Ø', 'Ø', 'Ø', 'Ø']

After Positive Example 1:

['Sunny', 'Warm', 'Normal', 'Strong', 'Warm', 'Same']

After Positive Example 2:

['Sunny', 'Warm', '?', 'Strong', 'Warm', 'Same']

After Positive Example 4:

['Sunny', 'Warm', '?', 'Strong', '?', '?']

FINAL MOST SPECIFIC HYPOTHESIS

Sky: Sunny

AirTemp: Warm

Humidity: ?

Wind: Strong

Water: ?

Forecast: ?#

2. CANDIDATE-ELIMINATION ALGORITHM

Google Colab

=====

import pandas as pd

import itertools

Step 1: Create CSV Dataset

data = {

"Sky": ["Sunny", "Sunny", "Rainy", "Sunny"],

"AirTemp": ["Warm", "Warm", "Cold", "Warm"],

"Humidity": ["Normal", "High", "High", "High"],

"Wind": ["Strong", "Strong", "Strong", "Strong"],

"Water": ["Warm", "Warm", "Warm", "Cool"],

"Forecast": ["Same", "Same", "Change", "Change"],

"EnjoySport": ["Yes", "Yes", "No", "Yes"]

}

df = pd.DataFrame(data)

Save dataset as CSV

```

        df.to_csv(
            "training_data.csv",
            index=False
        )

    print("CSV FILE CREATED:")
    print("training_data.csv")

# -----
# Step 2: Read CSV File
# -----

df = pd.read_csv("training_data.csv")

    print("\nTRAINING DATA")
    print("=" * 60)
    print(df)

# -----
# Step 3: Prepare Data
# -----

attributes = list(df.columns[:-1])

    domains = {
        attribute: list(
            df[attribute].unique()
        )
        for attribute in attributes
    }

    # Most specific hypothesis
    S = ["0"] * len(attributes)

    # Most general hypothesis
    G = ["?"] * len(attributes)

# -----
# Step 4: Helper Functions
# -----

```

```
def covers(hypothesis, example):

    for h, x in zip(hypothesis, example):

        if h == "∅":
            return False

        if h != "?" and h != x:
            return False

    return True


def more_general_or_equal(h1, h2):

    for a, b in zip(h1, h2):

        if a != "?" and a != b:
            return False

    return True


def remove_more_specific(G):

    result = []

    for g in G:

        keep = True

        for other in G:

            if g != other and more_general_or_equal(other, g):
                keep = False
                break

        if keep:
            result.append(g)

    return result
```

```

# -----
# Step 5: Candidate-Elimination Algorithm
# -----

for index, row in df.iterrows():

    example = list(
        row[attributes]
    )

    label = row["EnjoySport"]

# -----
# Positive Example
# -----

if label == "Yes":

    # Remove G hypotheses that do not cover example
    G = [
        g for g in G
        if covers(g, example)
    ]

    # Generalize S
    for i in range(len(attributes)):

        if S[i] == "∅":
            S[i] = example[i]

        elif S[i] != example[i]:
            S[i] = "?"

    # Remove G inconsistent with S
    G = [
        g for g in G
        if more_general_or_equal(g, S)
    ]

# -----
# Negative Example
# -----

```

```

        else:

            new_G = []

            for g in G:

                if covers(g, example):

                    # Specialize G
                    for i in range(len(attributes)):

                        if g[i] == "?":

                            for value in domains[attributes[i]]:

                                if value != example[i]:

                                    new_hypothesis = g.copy()

                                    new_hypothesis[i] = value

                                    # Must remain more general than S
                                    if more_general_or_equal(
                                        new_hypothesis,
                                        S
                                    ):
                                        new_G.append(
                                            new_hypothesis
                                        )

                                else:
                                    new_G.append(g)

            G = new_G

        # Remove redundant hypotheses
        G = remove_more_specific(G)

        print(
            f"\nAfter Example {index + 1}:"
        )

```

```

        print("S =", S)

        print("G =")

        for g in G:
            print(g)

# -----
#           # Step 6: Final Version Space
# -----

        print("\nFINAL VERSION SPACE")
        print("=" * 60)

        print("\nSpecific Boundary (S):")
        print(S)

        print("\nGeneral Boundary (G):")

        for g in G:
            print(g)

        print("\nAll hypotheses between S and G represent")
        print("the set of hypotheses consistent with the training data.")

```

[7]

0s

```

# =====

# 2. CANDIDATE-ELIMINATION ALGORITHM

# Google Colab

# =====

```



```
import pandas as pd

import itertools

# -----

# Step 1: Create CSV Dataset

# -----

data = {

    "Sky": ["Sunny", "Sunny", "Rainy", "Sunny"],

    "AirTemp": ["Warm", "Warm", "Cold", "Warm"],

    "Humidity": ["Normal", "High", "High", "High"],

    "Wind": ["Strong", "Strong", "Strong", "Strong"],

    "Water": ["Warm", "Warm", "Warm", "Cool"],

    "Forecast": ["Same", "Same", "Change", "Change"],

    "EnjoySport": ["Yes", "Yes", "No", "Yes"]

}
```

```
df = pd.DataFrame(data)
```

```
# Save dataset as CSV
```

```
df.to_csv(
```

```
"training_data.csv",
```

```
index=False
```

```
)
```

```
print("CSV FILE CREATED:")
```

```
print("training_data.csv")
```

```
# -----
```

```
# Step 2: Read CSV File
```

```
# -----
```

```
df = pd.read_csv("training_data.csv")
```

```
print("\nTRAINING DATA")

print("=" * 60)

print(df)

# -----

# Step 3: Prepare Data

# -----

attributes = list(df.columns[:-1])

domains = {

    attribute: list(

        df[attribute].unique()

    )

    for attribute in attributes

}
```

```
        # Most specific hypothesis

        S = ["∅"] * len(attributes)

    # Most general hypothesis

    G = [["?"] * len(attributes)]

# -----

    # Step 4: Helper Functions

# -----

    def covers(hypothesis, example):

        for h, x in zip(hypothesis, example):

            if h == "∅":

                return False
```

```
    if h != "?" and h != x:
```

```
        return False
```

```
    return True
```

```
def more_general_or_equal(h1, h2):
```

```
    for a, b in zip(h1, h2):
```

```
        if a != "?" and a != b:
```

```
            return False
```

```
    return True
```

```
def remove_more_specific(G):
```

```
        result = []

        for g in G:

            keep = True

            for other in G:

                if g != other and more_general_or_equal(other, g):

                    keep = False

                    break

            if keep:

                result.append(g)

        return result
```

```
# -----

# Step 5: Candidate-Elimination Algorithm

# -----

for index, row in df.iterrows():

    example = list(

        row[attributes]

    )

    label = row["EnjoySport"]

# -----

# Positive Example

# -----

if label == "Yes":
```

```
# Remove G hypotheses that do not cover example
```

```
G = [
```

```
    g for g in G
```

```
    if covers(g, example)
```

```
]
```

```
# Generalize S
```

```
for i in range(len(attributes)):
```

```
    if S[i] == "∅":
```

```
        S[i] = example[i]
```

```
    elif S[i] != example[i]:
```

```
        S[i] = "?"
```

```
# Remove G inconsistent with S
```

```
G = [
```



```
        g for g in G

        if more_general_or_equal(g, S)

    ]

# -----

# Negative Example

# -----

else:

    new_G = []

    for g in G:

        if covers(g, example):

            # Specialize G

            for i in range(len(attributes)):
```

```
        if g[i] == "?":

            for value in domains[attributes[i]]:

                if value != example[i]:

                    new_hypothesis = g.copy()

                    new_hypothesis[i] = value

                    # Must remain more general than S

                    if more_general_or_equal(

                        new_hypothesis,

                        S

                    ):

                        new_G.append(

                            new_hypothesis
```

```
)

    else:

        new_G.append(g)

    G = new_G

    # Remove redundant hypotheses

    G = remove_more_specific(G)

    print(

        f"\nAfter Example {index + 1}:"

    )

    print("S =", S)

    print("G =")
```

```
for g in G:

    print(g)

# -----

# Step 6: Final Version Space

# -----

print("\nFINAL VERSION SPACE")

print("=" * 60)

print("\nSpecific Boundary (S):")

print(S)

print("\nGeneral Boundary (G):")

for g in G:

    print(g)
```

```
print("\nAll hypotheses between S and G represent")

print("the set of hypotheses consistent with the training data.")
```

CSV FILE CREATED:
training_data.csv

TRAINING DATA

	Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
0	Sunny	Warm	Normal	Strong	Warm	Same	Yes
1	Sunny	Warm	High	Strong	Warm	Same	Yes
2	Rainy	Cold	High	Strong	Warm	Change	No
3	Sunny	Warm	High	Strong	Cool	Change	Yes

After Example 1:

```
S = ['Sunny', 'Warm', 'Normal', 'Strong', 'Warm', 'Same']
G = ['?', '?', '?', '?', '?', '?']
```

After Example 2:

```
S = ['Sunny', 'Warm', '?', 'Strong', 'Warm', 'Same']
G = ['?', '?', '?', '?', '?', '?']
```

After Example 3:

```
S = ['Sunny', 'Warm', '?', 'Strong', 'Warm', 'Same']
G = ['Sunny', '?', '?', '?', '?', '?']
  ['?', 'Warm', '?', '?', '?', '?']
  ['?', '?', '?', '?', '?', 'Same']
```

After Example 4:

```
S = ['Sunny', 'Warm', '?', 'Strong', '?', '?']
G = ['Sunny', '?', '?', '?', '?', '?']
  ['?', 'Warm', '?', '?', '?', '?']
```

FINAL VERSION SPACE

Specific Boundary (S):

```
['Sunny', 'Warm', '?', 'Strong', '?', '?']
```

General Boundary (G):

```
['Sunny', '?', '?', '?', '?', '?']
['?', 'Warm', '?', '?', '?', '?']
```

All hypotheses between S and G represent
the set of hypotheses consistent with the training data.

```

# =====

# 3. ID3 DECISION TREE ALGORITHM
# Google Colab

# =====

import pandas as pd
import numpy as np

from sklearn.tree import DecisionTreeClassifier
from sklearn.preprocessing import LabelEncoder
from sklearn.tree import plot_tree

import matplotlib.pyplot as plt

# -----

# Step 1: Create Dataset

# -----

data = {
    "Outlook": [
        "Sunny", "Sunny", "Overcast", "Rain",
        "Rain", "Rain", "Overcast", "Sunny",
        "Sunny", "Rain", "Sunny", "Overcast",
        "Overcast", "Rain"
    ],

    "Temperature": [
        "Hot", "Hot", "Hot", "Mild",
        "Cool", "Cool", "Cool", "Mild",
        "Cool", "Mild", "Mild", "Mild",
        "Hot", "Mild"
    ],

    "Humidity": [
        "High", "High", "High", "High",
        "Normal", "Normal", "Normal", "High",
        "Normal", "Normal", "Normal", "High",
        "Normal", "High"
    ],

    "Wind": [
        "Weak", "Strong", "Weak", "Weak",

```

```

        "Weak", "Strong", "Strong", "Weak",
        "Weak", "Weak", "Strong", "Strong",
            "Weak", "Strong"
        ],

        "Play": [
            "No", "No", "Yes", "Yes",
            "Yes", "No", "Yes", "No",
            "Yes", "Yes", "Yes", "Yes",
            "Yes", "No"
        ]
    }

df = pd.DataFrame(data)

print("PLAY TENNIS DATASET")
print("=" * 60)
print(df)

# -----
# Step 2: Calculate Entropy
# -----

def entropy(y):

    values, counts = np.unique(
        y,
        return_counts=True
    )

    probabilities = counts / len(y)

    return -np.sum(
        probabilities * np.log2(probabilities)
    )

# -----
# Step 3: Calculate Information Gain
# -----

def information_gain(df, attribute, target):

```

```

        total_entropy = entropy(
            df[target]
        )

        values, counts = np.unique(
            df[attribute],
            return_counts=True
        )

        weighted_entropy = 0

        for value, count in zip(values, counts):

            subset = df[
                df[attribute] == value
            ]

            weighted_entropy += (
                count / len(df)
            ) * entropy(
                subset[target]
            )

        return total_entropy - weighted_entropy

# -----
# Step 4: Calculate Information Gain
# -----

        attributes = [
            "Outlook",
            "Temperature",
            "Humidity",
            "Wind"
        ]

        print("\nINFORMATION GAIN")
        print("=" * 60)

        for attribute in attributes:

```



```

        gain = information_gain(
            df,
            attribute,
            "Play"
        )

        print(
            f"{attribute}: {gain:.4f}"
        )

# -----
# Step 5: Encode Categorical Data
# -----

encoded_df = df.copy()

encoders = {}

for column in encoded_df.columns:

    encoder = LabelEncoder()

    encoded_df[column] = encoder.fit_transform(
        encoded_df[column]
    )

    encoders[column] = encoder

# -----
# Step 6: Separate Features and Target
# -----

X = encoded_df[
    attributes
]

y = encoded_df["Play"]

# -----
# Step 7: Build ID3 Decision Tree

```

```
# -----  
  
    model = DecisionTreeClassifier(  
        criterion="entropy",  
        random_state=42  
    )  
  
    model.fit(  
        X,  
        y  
    )  
  
# -----  
# Step 8: Display Tree  
# -----  
  
plt.figure(figsize=(15, 8))  
  
    plot_tree(  
        model,  
        feature_names=attributes,  
        class_names=encoders["Play"].classes_,  
        filled=True,  
        rounded=True  
    )  
  
    plt.title(  
        "ID3 Decision Tree"  
    )  
  
    plt.show()  
  
# -----  
# Step 9: Classify New Sample  
# -----  
  
new_sample = pd.DataFrame({  
    "Outlook": ["Sunny"],  
    "Temperature": ["Cool"],  
    "Humidity": ["High"],  
    "Wind": ["Strong"]  
})
```

```

    })

    # Encode new sample using training encoders

    for column in attributes:

        new_sample[column] = encoders[column].transform(
            new_sample[column]
        )

    # Prediction

    prediction = model.predict(
        new_sample
    )

    predicted_class = encoders[
        "Play"
    ].inverse_transform(
        prediction
    )

# -----
# Step 10: Display Prediction
# -----

print("\nNEW SAMPLE")
print("=" * 60)

print(
    "Outlook: Sunny"
)

print(
    "Temperature: Cool"
)

print(
    "Humidity: High"
)

```

```

        print(
            "Wind: Strong"
        )

    print("\nPREDICTION")
    print("=" * 60)

    print(
        "Play Tennis:",
        predicted_class[0]
    )

```

PLAY TENNIS DATASET

```

=====
      Outlook Temperature Humidity  Wind Play
0   Sunny      Hot    High   Weak  No
1   Sunny      Hot    High  Strong  No
2  Overcast    Hot    High   Weak  Yes
3    Rain     Mild    High   Weak  Yes
4    Rain     Cool   Normal   Weak  Yes
5    Rain     Cool   Normal  Strong  No
6  Overcast    Cool   Normal  Strong  Yes
7   Sunny     Mild    High   Weak  No
8   Sunny     Cool   Normal   Weak  Yes
9    Rain     Mild   Normal   Weak  Yes
10  Sunny     Mild   Normal  Strong  Yes
11 Overcast    Mild    High  Strong  Yes
12 Overcast    Hot    Normal   Weak  Yes
13  Rain      Mild    High  Strong  No

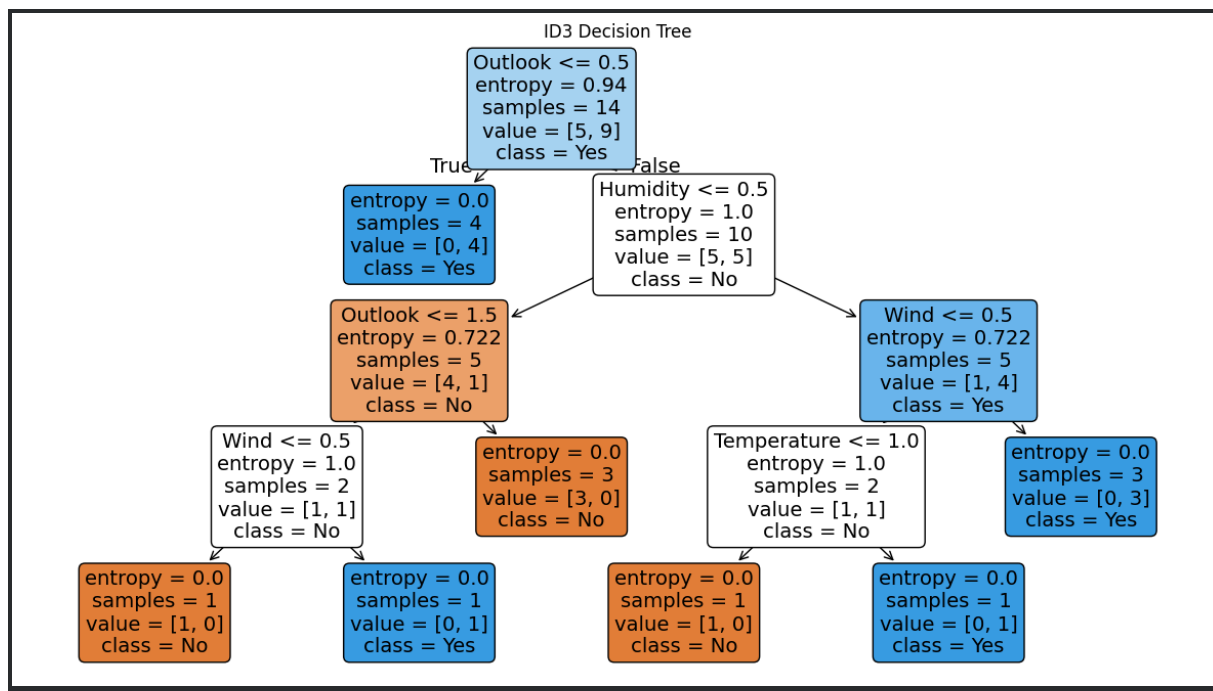
```

INFORMATION GAIN

```

=====
      Outlook: 0.2467
      Temperature: 0.0292
      Humidity: 0.1518
      Wind: 0.0481

```



NEW SAMPLE

=====

Outlook: Sunny
Temperature: Cool
Humidity: High
Wind: Strong

PREDICTION

=====

Play Tennis: No